DSC 40B Discussion 3 Worksheet Solutions

Friday, April 17, 2026

Imagine you are a tutor, and you need to explain something to someone who has only a vague understanding of what's going on. Do your best to get the point across. **ALTERNATE**.

If you're not sure, ask your peer if they know how to explain it, then REPEAT the explanation back. Remember, the point of the exercise is to **vocalize** your thoughts.

If you are both unsure, make sure to come to/tutor for help! Do not use ChatGPT or any other LLM! The point of discussion is to prepare you for the exams.

Have one person in the pair open the Discussion 1 Gradescope assignment. After both you **and** your partner finish a problem **and explain it**, answer the corresponding multiple choice/short answer questions for both versions.. You will receive instant feedback on the questions, whether you are correct or not. If your answer is incorrect, please go back to the question to understand it and feel free to ask tutors about anything you are confused about. Once you have finished the entire worksheet or discussion is ending, turn in your assignment and add your partner to the submission.

Quick Icebreaker

Introduce yourself to your partner by sharing name, year, major, etc. What is your favorite study spot on campus? (Do not spend more than 2 minutes!)

Questions

Question 1:

First Person:

Find the expected time complexity of the code below, where `nums` is a list of n unique integers from 1 to n in random order. Show your work and explain your reasoning.

```
def scan_until_match(nums): # nums comprises of n unique integers
    for i in range(n):
        if nums[i] == 1:
            return "Kaboom!"
    return "No Kaboom."
```

$$E = \frac{1}{n}\sum_{i=1}^{n} i = \frac{1}{n}(1 + 2 + \ldots + n - 1 + n) \rightarrow \Theta(n)$$

Second Person:

Find the expected time complexity of the code below, where `nums` is a list of n integers containing exactly 5 1's. Show your work and explain your reasoning.

```
def sample_until_match(nums): # nums comprises of 5 1's.
    n = len(nums)
    count = 0

    while True:
        idx = random.randrange(n)  # uniform random index in [0, n-1]
        if nums[idx] == 1:
            return "Kaboom!"
```

The probability to hit at k-th iteration: $P(X = k) = (1 - \frac{5}{n})^{k-1}(\frac{5}{n})$

$\rightarrow E = \sum_{k=1}^{\infty} P(X = k)k = \sum_{k=1}^{\infty} (1 - \frac{5}{n})^{k-1}(\frac{5}{n}) \rightarrow \Theta(n)$*, from geometric sum formula*

Question 2:

Second Person:

Given a list of n integers, find the largest sum that can be produced by three integers in the list. Provide a theoretical lower bound. Explain your reasoning.

$\Theta(n)$***; Must iterate through all numbers to know the three largest ones.***

First Person:

Given a sorted list of n integers, find the largest sum that can be produced by three integers in the list. Provide a theoretical lower bound. Explain your reasoning.

$\Theta(1)$*; The three largest numbers are just the three at the end (or beginning). Therefore always a constant amount of checks to guarantee the correct answer.*

Question 3:

First Person:

Given a sorted list of n unique integers and a target integer t, find the index of t, which **is guaranteed** to exist in the list. Provide a theoretical lower bound. Explain your reasoning.

Perform binary search, which gives a log(n) guarantee.

Second Person:

Given a sorted list of n unique integers and a target integer t, find the index of t, which **is not guaranteed** to exist in the list. Provide a theoretical lower bound. Explain your reasoning.

$\theta(logn)$*; You can still perform binary search without the guaranteed existence of the target.*

Question 4:

Second Person:

You are given two unsorted lists of n integers, list A and B. For every element in A, find how many elements in B are larger than or equal to it. Provide a theoretical lower bound. Explain your reasoning.

$\theta(n^2)$*; For every element in A, must scan all n elements in B to know the exact number of elements larger. Performs this n times, therefore quadratic time*

First Person:

You are given two lists of n integers, list A and B. List A is unsorted and list B is sorted. For every element in A, find how many elements in B are larger than or equal to it. Provide a theoretical lower bound. Explain your reasoning.

$\Theta(nlogn)$*; For every element in A, can perform binary search on B to know where it would be inserted and therefore how many elements larger than it. This is log(n) time per element in A. Therefore lower bound for all elements in A is nlog(n).*

Question 5:

Suppose binary search is performed on the following array using the implementation covered in lecture.

$[1, 4, 7, 8, 8, 10, 15, 51, 60, 65, 71, 72, 101]$

First Person:

What numbers can be found running `arr[middle]==target` 3 times? Explain your reasoning. If it is not possible to run 3 times, explain why.

Three partitions of binary code will yield the target to be one of 4, 8, 60, or 72.

Second Person:

What numbers can be found running `arr[middle]==target` 5 times? Explain your reasoning. If it is not possible to run 5 times, explain why.

The maximum possible number of equality comparisons is 4, which is for when the target is not present in the list.

Question 6:

Second Person:

State (but do not solve) the recurrence relation describing the function's run time. Explain your reasoning.

```
def foo(n):
    if n <= 2:
        return
    for i in range(n):
        for j in range(i, n):
            print(i)
    return foo(n//2) + foo(n//2)
```

$T(n) = 2T(n/2) + n^2$
$T(0, 1, 2) = 1$

First Person:

State (but do not solve) the recurrence relation describing the function's run time. Explain your reasoning.

```
def fi(n):
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fi(n - 1) + fi(n - 2)
```

$T(n) = T(n - 2) + T(n - 1)$
$T(0, 1) = 1$

Question 7:

First Person:

Solve the following recurrence relation. Show your work and explain your reasoning.

$T(n) = T(n - 1) + n$

$T(0) = 0$

$T(n) = T(n-1) + n = T(n-2) + n - 1 + n = 1 + 2 + 3 + \ldots + n - 1 + n \rightarrow \Theta(n2)$

Second Person:

Solve the following recurrence relation. Show your work and explain your reasoning.

$$T(n) = 4T(\tfrac{n}{4}) + n$$

$$T(1) = 1$$

$T(n) = 4T(n/4) + n)$
$= 4(4T(n/16) + n/4)) + n$
$= 4(4T(n/16) + n/4)) + n$
$= 16T(n/16) + 2n$
$\rightarrow T(n) = 4kT(n/4k) + kn$
$\rightarrow T(n) = 4\,k\,T(n/4\,k\,) + kn,\ for\ k - th\ iteration$

Since there are total of $k = log(n)$ *iterations,*
$T(n) = 4log(n)T(n/4log(n)) + log(n)*nT(n)$
$= 4\,log(n)\,T(n/4\,log(n)\,) + log(n)*n$
$\rightarrow T(n) = log(n)*\Theta(1) + log(n)*n \rightarrow \Theta(nlogn)$

Question 8:

Second Person:

Which of the following lists could have resulted from completing 2 steps of selection sort? Select all that apply and explain your reasoning.

- ☐ $[9, 10, 1, 2, 3, 4]$
- ☑ ~~[1, 2, 9, 10, 3, 4]~~
- ☑ ~~[1, 2, 3, 4, 9, 10]~~
- ☐ $[9, 1, 2, 3, 4, 10]$
- ☐ $[4, 3, 2, 1, 9, 10]$

First Person:

Which of the following lists could have resulted from completing 2 steps of insertion sort? Select all that apply and explain your reasoning.

- ☑ ~~[9, 10, 1, 2, 3, 4]~~
- ☑ ~~[1, 2, 9, 10, 3, 4]~~
- ☑ ~~[1, 2, 3, 4, 9, 10]~~
- ☐ $[9, 1, 2, 3, 4, 10]$
- ☐ $[4, 3, 2, 1, 9, 10]$

Question 9:

First Person:

Which of the following describes a loop invariant associated with the following program? Explain your reasoning.

```
def function(lst):
    mystery = 0
    for num in lst:
        mystery = max(mystery, num)
    return mystery
```

1. After α iterations of the loop, mystery stores the mean of the first α numbers in lst
2. After α iterations of the loop, mystery stores the standard deviation of the first α numbers in lst
3. After α iterations of the loop, mystery stores the median of the first α numbers in lst
4. After α iterations of the loop, mystery stores the maximum of the first α numbers in lst

Choice 4

Second Person:

Which of the following describes a loop invariant associated with the following program? Explain your reasoning.

```
def function(lst):
    mystery = 0
    count = 0
    for num in lst:
        mystery = (mystery * count + num) / (count + 1)
        count += 1
    return mystery
```

1. After α iterations of the loop, mystery stores the mean of the first α numbers in lst
2. After α iterations of the loop, mystery stores the standard deviation of the first α numbers in lst
3. After α iterations of the loop, mystery stores the median of the first α numbers in lst
4. After α iterations of the loop, mystery stores the maximum of the first α numbers in lst

Choice 1